

# install\_git\_hooks

## scripts/install\_git\_hooks.sh and scripts/git\_hooks/ — the tracked git hooks

**Revision:** 1 **Last modified:** 2026-09-26T16:33:51Z **Status:** active **Item:** BOB-251 (pre-push hook failed silently when its temporary worktree could not be created)

### Overview

The project keeps its git hooks as tracked files in `scripts/git_hooks/` and installs them with `scripts/install_git_hooks.sh`, which copies each of them into `.git/hooks/` and makes it executable. `core.hooksPath` is not used, so what git actually runs is the **installed copy**, not the tracked source. A change to a hook source reaches a checkout only when the installer is run in that checkout.

| Hook       | What it does                                                                                                                                                                                                                                         |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pre-commit | §11.4.75: refuses a commit that stages a mutation marker into a first-party production source file (download-proxy/, qBitTorrent-go/, scripts/, plugins/, webui-bridge.py; .sh/.bash/.py/.go; tests, docs, QA evidence and vendored trees excluded). |
| pre-push   | §11.4.75 at push time: runs the pre-commit rules against the content being pushed, so a                                                                                                                                                              |

| Hook                     | What it does                                                                               |
|--------------------------|--------------------------------------------------------------------------------------------|
|                          | marker committed with <code>--no-verify</code> is still caught.                            |
| <code>commit-msg</code>  | Requires a <code>Bypass-rationale:</code> footer when a bypass marker file is present.     |
| <code>post-commit</code> | Appends a line per commit to the \$11.4.75 audit log and runs the forensic capture helper. |

## Prerequisites

- bash, git, df.
- Run from inside the repository (the installer resolves the repository root with `git rev-parse --show-toplevel`).

## Usage examples

Install or refresh every hook in the current checkout:

```
bash scripts/install_git_hooks.sh
```

Confirm that the installed copies match the tracked sources:

```
for h in pre-commit pre-push commit-msg post-commit; do
  cmp -s "scripts/git_hooks/$h" ".git/hooks/$h" && echo "$h
    current" || echo "$h STALE"
done
```

A refresh is needed after any commit that changes a file in `scripts/git_hooks/`. The installer overwrites the four hooks it manages and nothing else; it creates the `bypass-audit` and `audit-log` files only if they do not exist.

## How the pre-push check works

git calls `pre-push <remote> <url>` with one line per ref on stdin: `<local ref> <local sha> <remote ref> <remote sha>`. For each line the hook:

1. skips deletions (a local sha of all zeros);
2. lists the commits being pushed — `<remote sha>..<local sha> when the remote tip is known locally, otherwise everything reachable`

from the local sha that is not on any remote-tracking ref (new branches);

3. collects the files those commits add, copy, modify or rename;
4. hands that list to the installed pre-commit hook in revision mode (BOBA\_HOOK\_CHECK\_REV=<local sha>, file list on stdin), which reads each file from the pushed tip with `git ls-tree / git show` and applies the same scope rules and patterns it applies at commit time.

A marker that was added and removed again before the pushed tip does not block; a file deleted by a later pushed commit is skipped.

The hook reads git objects only. It creates no worktree, no index and no temporary file, so a full or read-only `$TMPDIR` cannot affect it.

## Exit codes

| Code | Meaning                                                                                                                                                                          |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Nothing to object to; the push proceeds.                                                                                                                                         |
| 1    | Marker(s) found in content being pushed; each file and line is listed.                                                                                                           |
| 2    | A step could not complete. The push is refused and the report names the step, prints the git error, and shows the free space of the git directory and of <code>\$TMPDIR</code> . |

Example of a failure report:

```
pre-push: FAILED at step 'list commits to push (...)' – the push was
NOT checked and is refused.
```

```
git error: error: unable to write file
```

```
free space:
```

```
git dir /home/.../.git: 120G free of 900G (87% used, mount /
home)
```

```
  TMPDIR /tmp: 6.1G free of 16G (61% used, mount /tmp)
```

```
Remediation: free disk space or fix the git error above, then
push again.
```

## Edge cases

- **pre-commit not installed.** The pre-push hook prints a note naming the installer and lets the push proceed (unchanged behaviour, now visible).
- **Only one of the two hooks refreshed.** Revision mode lives in pre-commit; an old pre-commit ignores BOBA\_HOOK\_CHECK\_REV and checks the staged set instead. The installer always installs both together.
- **Remote tip not present locally** (for example the remote moved and the push will be rejected as non-fast-forward): the hook falls back to “not on any remote-tracking ref”, which checks at least as much.
- **Path names containing newlines** are not supported by either hook (the file lists are newline-separated).

## Internal behaviour — what changed in BOB-251

The previous pre-push created `git worktree add "$WORKTREE" HEAD` in a `mktemp -d` directory, with all output discarded, under `set -e`. When `/tmp` was full the worktree could not be written (unable to write file, Could not reset index file to revision HEAD), the hook stopped without a word, and git printed only failed to push some refs; three pushes failed this way on 2026-09-26. The same hook ran pre-commit inside the fresh worktree, where nothing is staged, so the \$11.4.75 check it claimed to enforce passed for every push: a marker committed with `--no-verify` was pushed without objection.

## Anti-bluff verification

`tests/hooks/test_pre_push_hook.sh` builds throwaway repositories whose only remote is a local bare repository, installs the hooks with a copy of the installer, and pushes for real:

- the installer produces a byte-identical, executable `.git/hooks/pre-push`;
- a clean push succeeds (control); a docs-only marker and a marker removed before the tip do not block;
- a production-file marker committed with `--no-verify` blocks the push, also on a brand-new branch; deletions pass;
- an unwritable `$TMPDIR` does not break a healthy push;

- with a `git` shim that fails like a full disk, a failing `git worktree add` cannot break the hook, and failing object reads give a named, non-zero, disk-space-carrying report instead of silence.

Against the previous hook sources it fails 6 of 12 checks (silent failures and the vacuous check); against the current sources it passes 12 of 12. Three paired mutations (silencing the failure report, dropping revision mode, treating an unreadable file as absent) each turn it red. `tests/hooks/test_pre_commit_mutation_scope.sh` covers the pre-commit scope rules and still passes 7 of 7.

## Related scripts

- `scripts/commit-push-all.sh` — its push stage runs through this hook (`docs/scripts/commit-push-all.md`).
- `scripts/pre_build/check_cm_no_production_mutation_residue.sh` — the pre-build gate using the same scope technique.
- `scripts/hooks/auto-commit-forensic-capture.sh` — called by post-commit.

**Last verified:** 2026-09-26 (hook tests 12/12 and 7/7 in `mktemp` sandboxes).